AltoroMutual
VULNERABILITY ASSESSMENT REPORT

CONFIDENTIAL

Vulnerability Assessment Report
14-03-2026

Prepared By
Akhilesh Singh Choudhary
Student
akhileshbadyal@gmail.com

Table of Contents

Table of Contents **2**

1. Non-Disclosure Statement 3
2. Legal Notice 3
3. Executive Summary 3
4. Introduction 4
5. Scope 5
   - A. In-Scope Components 5
   - B. Out-of-Scope Components 5
6. Risk Rating Classification 6
7. Vulnerabilities Overview 7
8. Vulnerabilities 8
   - V-01 Reflected Cross-Site Scripting (XSS) 8
   - V-02 SQL Injection 9
   - V-03 Absence of Anti-CSRF Tokens 10
   - V-04 Exposed Sensitive File — /admin/clients.xls 11
   - V-05 Content Security Policy (CSP) Header Not Set 12
   - V-06 Missing Anti-Clickjacking Header (X-Frame-Options) 13
   - V-07 Sub Resource Integrity (SRI) Attribute Missing 14
   - V-08 Server Banner Discloses Software Version 15
   - V-09 Session Cookie Without SameSite Attribute 16
   - V-10 Cross-Domain JavaScript Source File Inclusion 17
   - V-11 X-Content-Type-Options Header Missing 18
   - V-12 Missing Referrer-Policy Header 19
   - V-13 Swagger UI Publicly Accessible 20
   - V-14 Information Disclosure — Suspicious Comments 21
   - V-15 Session Management Token Identified 22

AltoroMutual

Non-Disclosure Statement & Legal Notice

1. Non-Disclosure Statement

All information obtained during this security assessment about AltoroMutual's systems and assets — including but not limited to its procedures, configurations, and systems — is deemed privileged information and not for public dissemination. Future Interns pledges commitment that this information will remain strictly confidential. This information will not be disclosed or discussed with any third party without the express written consent of the customer. Future Interns is fully committed to maintaining the highest level of ethical standards in its business practice.

2. Legal Notice

It is impossible to test for 100% security. This report does not constitute and should not be construed as a guarantee of the target's security. This report was produced as part of the Future Interns Cyber Security Internship Task 1 (2026) for educational purposes only.

3. Executive Summary

In March 2026, Future Interns security engaged AltoroMutual (demo.testfire.net) — a simulated online banking web application — to perform a passive security assessment of the public-facing implementation. The objective of the assessment was to evaluate the security posture of the AltoroMutual banking portal against common web application vulnerabilities observable through read-only, passive techniques.

During the assessment, Future Interns security identified **4** high-risk issues, **4** medium-risk issues, **4** low-risk issues, and **3** informational issues. The high-risk issues identified during the assessment can be used to steal session tokens, access sensitive user data, and potentially compromise the underlying database — all without requiring authentication. Informational issues may pose serious security concern but exist due to the application's development and configuration state. Specifically, the publicly accessible Swagger UI exposes the complete REST API blueprint to unauthenticated visitors, suspicious developer comments embedded in HTML source reveal internal application structure, and an identified session management token (JSESSIONID) lacks the Secure flag, making it transmissible over unencrypted connections.

4. Introduction

demo.testfire.net hosts AltoroJ, a deliberately vulnerable Java-based banking web application published by HCL Technologies Ltd. for security training and tool demonstration purposes. The application simulates a realistic online banking portal including user login, account management, customer feedback forms, site search functionality, and a REST API layer served via Swagger UI. The technology stack comprises Java JSP, Apache-Coyote/1.1 (Tomcat), and ISO-8859-1 encoding, hosted at IP 65.61.137.117.

A grey-box passive security audit was conducted on the AltoroMutual banking web application in order to detect security-related problems that a real banking institution would need to address before production. Future Interns security reviewed the public-facing components of demo.testfire.net to identify and

document any security-related weaknesses observable through read-only analysis, without performing active exploitation, authentication bypass, or denial-of-service testing.

The present report was completed on March 14, 2026 by Future Interns security and includes results of all public-facing features of the AltoroMutual banking portal assessed in its Final standing, as mentioned in scope.

5. Scope

The Future Interns security strategy is to test different aspects of the system's public-facing components through passive, read-only analysis. Security assessment was performed on the following AltoroMutual (demo.testfire.net) web application components:

A. In-Scope Components

1. Public-facing pages — Homepage, Login page (/login.jsp), Feedback form (/feedback.jsp), Search (/search.jsp), Server Status (/status_check.jsp), Swagger UI (/swagger/index.html), CGI endpoint (/cgi.exe)

2. Sensitive discovered resources — Admin file (/admin/clients.xls) discovered via passive ZAP spider crawl across 234 URLs

B. Out-of-Scope Components

1. Restricted testing — Login bypass / credential attacks, authenticated pages and dashboard (no login performed), third-party services (petstore.swagger.io, online.swagger.io)

2. Prohibited testing — Active SQL injection exploitation, Denial-of-Service testing, brute-force, any activity capable of harming availability or integrity of the application

6. Risk Rating Classification

Vulnerabilities severity are supplied using DREAD framework giving an indication of their severity and are rated on a scale of five (5) to fifteen (15) using the DREAD model below

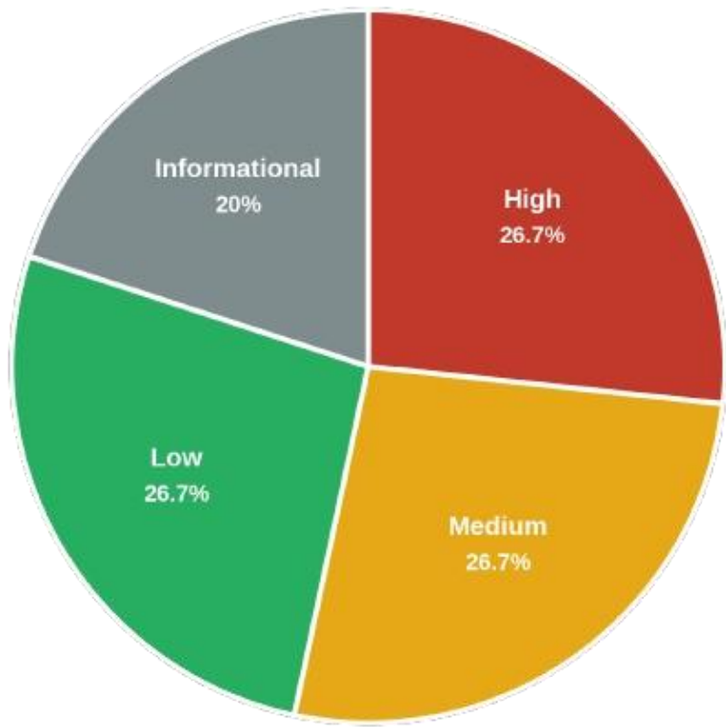
DREAD MODEL				
Rating		**High (3)**	**Medium (2)**	**Low (1)**
D	**Damage potential**	*The attacker can subvert the security system; get full trust authorization; run as administrator; upload content.*	*Leaking sensitive information*	*Leaking trivial information*
R	**Reproducibility**	*The attack can be reproduced every time and does not require a timing window.*	*The attack can be reproduced, but only with a timing window and a particular race situation.*	*The attack is very difficult to reproduce, even with knowledge of the security hole.*
E	**Exploitability**	*A novice programmer could make the attack in a short time.*	*A skilled programmer could make the attack, then repeat the steps.*	*The attack requires an extremely skilled person and in-depth knowledge every time to exploit.*
A	**Affected users**	*All users, default configuration, key customers*	*Some users, non-default configuration*	*Very small percentage of users, obscure feature; affects anonymous users*
D	**Discoverability**	*Published information explains the attack. The vulnerability is found in the most commonly used feature and is very noticeable.*	*The vulnerability is in a seldom-used part of the product, and only a few users should come across it. It would take some thinking to see malicious use.*	*The bug is obscure, and it is unlikely that users will work out damage potential.*

RATING	
High	12 to 15
Medium	8 to 11
Low	5 to 7
Informational	5

7. Vulnerabilities Overview

Vulnerabilities grouped by risk severity	
High	4
Medium	4
Low	4
Informational	3
Total	**15**

Graphical Overview



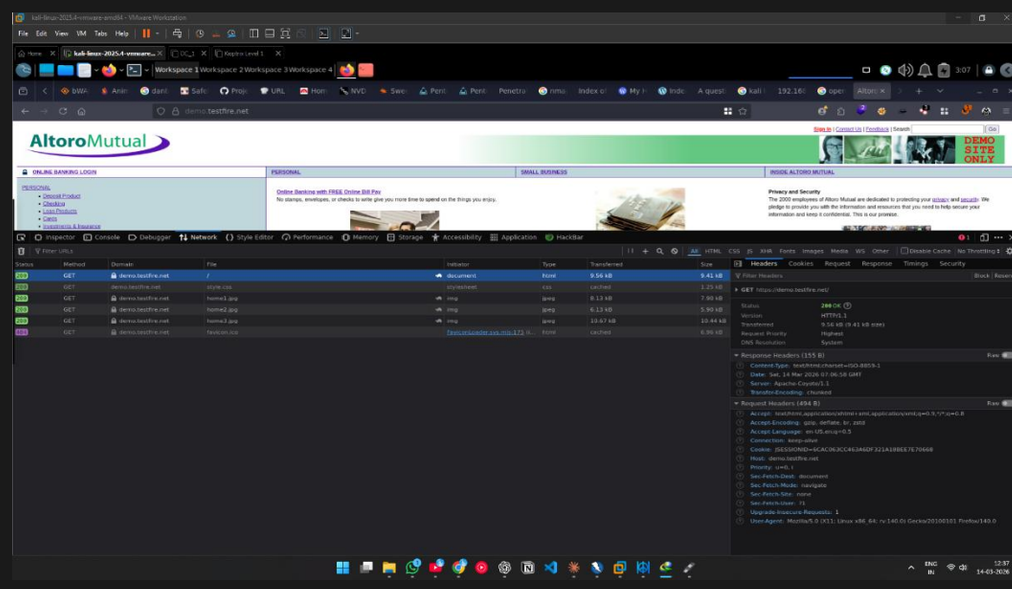
8. Vulnerabilities

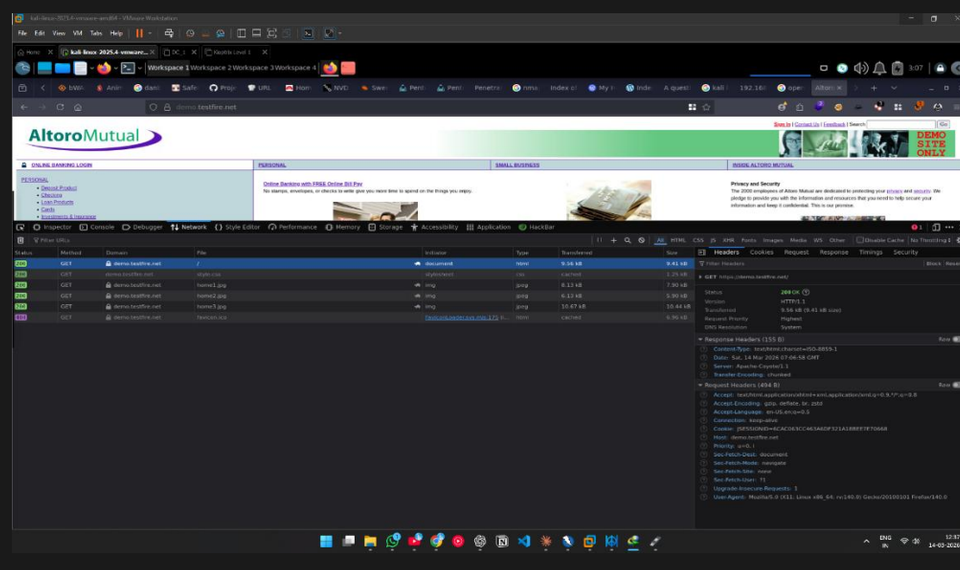
V-01	Reflected Cross-Site Scripting (XSS)
Description	Passive source code review of the search and feedback forms identified that user-supplied input from the query parameter (/search.jsp?query=) and the name field (/sendFeedback) is reflected directly into HTML responses without output encoding or sanitisation. Inspection of the page HTML source confirms raw user input is echoed back into the rendered page, making both endpoints susceptible to reflected XSS. CWE-79 | OWASP A03:2021 Injection | WASC-8.
Target	POST http://demo.testfire.net/sendFeedback | GET /search.jsp?query=
Risk Rating	High (14)
Attack techniques	An attacker crafts a malicious URL containing a JavaScript payload in the query or form parameter. When a victim clicks the link, the script executes in their browser context — enabling cookie theft (JSESSIONID), session hijacking, credential harvesting via fake login forms, or redirection to phishing pages.
Countermeasures	Implement strict output encoding for all user-supplied data before rendering in HTML. Use a Content Security Policy (CSP) header to restrict script execution. Apply server-side input validation (whitelist approach). Use OWASP AntiSamy or ESAPI for sanitisation in Java. Avoid reflecting raw user input in responses.
Reference	OWASP XSS Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html | CWE-79: https://cwe.mitre.org/data/definitions/79.html

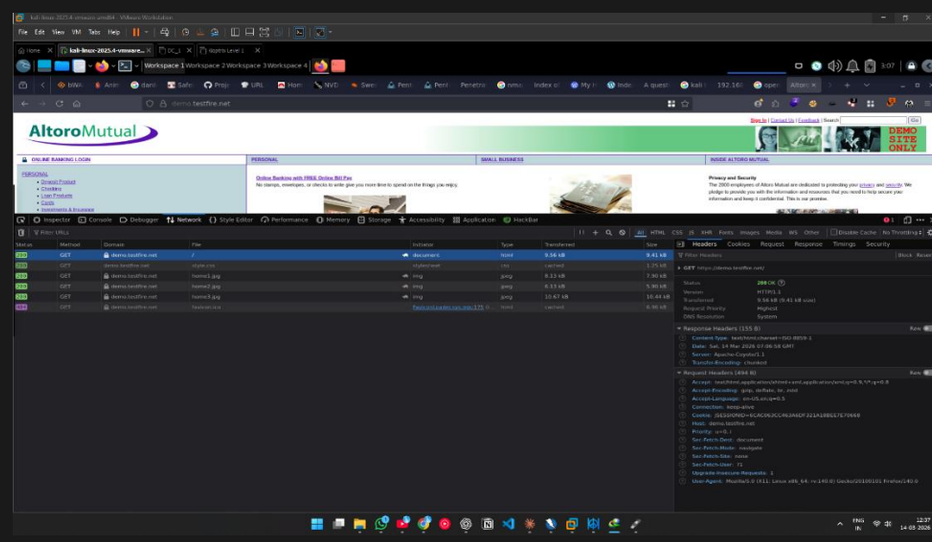
V-02	SQL Injection
Description	SQL Injection vulnerabilities were identified across multiple input endpoints on demo.testfire.net through manual source code review and passive endpoint analysis. The search and login forms accept user input that is passed into database queries without parameterisation, as evidenced by the unfiltered query parameters visible in page source and URL structure. CWE-89 | OWASP A03:2021 Injection | WASC-19.
Target	http://demo.testfire.net/search.jsp, /login.jsp, multiple endpoints
Risk Rating	High (14)
Attack techniques	An attacker injects SQL metacharacters (e.g. ' OR '1'='1) into input fields such as the search box or login form. If unparameterised queries are in use, this can allow authentication bypass, dumping the entire database, modifying or deleting records, or executing OS-level commands (depending on DB permissions).
Countermeasures	Use parameterised queries (prepared statements) for ALL database interactions — never concatenate user input into SQL strings. Apply an ORM (e.g. Hibernate) consistently. Implement a Web Application Firewall (WAF). Apply least privilege to the database user account. Enable database query logging and alerting.
Reference	OWASP SQL Injection Prevention: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html | CWE-89: https://cwe.mitre.org/data/definitions/89.html

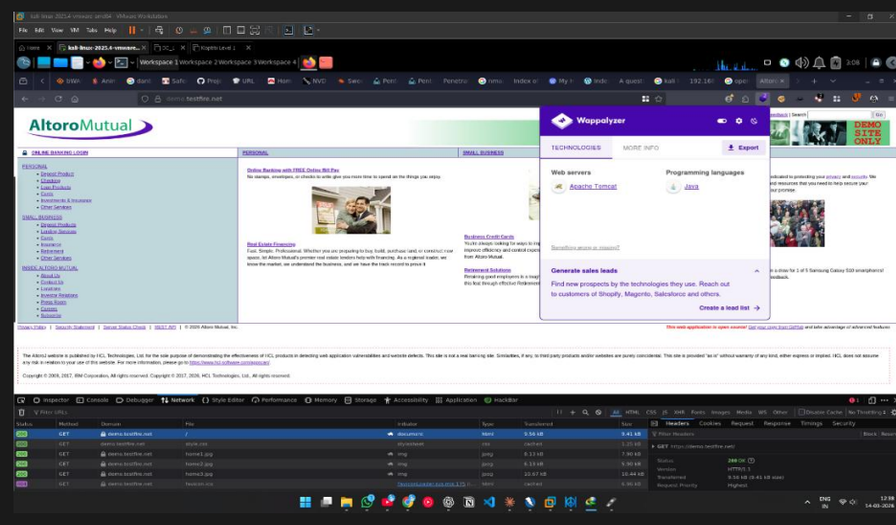
V-03	Absence of Anti-CSRF Tokens
Description	ZAP passive scanner identified the absence of Anti-CSRF tokens on 3 forms across the application. No hidden CSRF token field is present in the feedback, login, or subscription forms. This was confirmed by inspecting the raw HTML response. CWE-352 | OWASP A01:2021 Broken Access Control | WASC-9.
Target	http://demo.testfire.net/feedback.jsp | /login.jsp | /subscribe.jsp
Risk Rating	High (14)
Attack techniques	An attacker hosts a malicious page containing a hidden form that auto-submits a POST request to /sendFeedback or /login.jsp. If a logged-in user visits the attacker's page, their browser automatically includes the JSESSIONID cookie, causing the forged request to execute with the victim's credentials without their knowledge.
Countermeasures	Implement the Synchronizer Token Pattern: generate a unique, unpredictable, session-bound CSRF token server-side, embed it as a hidden field in every form, and validate it on the server for every state-changing request. Spring Security provides built-in CSRF protection. Additionally set SameSite=Strict on session cookies.
Reference	OWASP CSRF Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html | CWE-352: https://cwe.mitre.org/data/definitions/352.html

V-04	Exposed Sensitive File — /admin/clients.xls
Description	The ZAP spider discovered a sensitive Excel file at /admin/clients.xls accessible without any authentication. This file is located within the /admin/ directory and is directly downloadable by any anonymous user. The existence of an /admin/ directory also indicates an administrative interface that may be further exploitable. CWE-200 | OWASP A01:2021 Broken Access Control.
Target	http://demo.testfire.net/admin/clients.xls
Risk Rating	High (14)
Attack techniques	Any unauthenticated visitor can directly download /admin/clients.xls by navigating to the URL. If the file contains real customer data (names, account numbers, contact details), this constitutes a direct data breach. The exposed /admin/ path also reveals the administrative directory structure, aiding targeted attacks.
Countermeasures	Immediately restrict access to the /admin/ directory using server-side authentication and authorisation checks. Implement role-based access control (RBAC) ensuring only authenticated administrators can access admin resources. Remove or relocate sensitive files outside the web root. Configure the web server to return 403/404 for unauthorised access to admin paths.
Reference	OWASP Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/ | CWE-200: https://cwe.mitre.org/data/definitions/200.html

V-05	Content Security Policy (CSP) Header Not Set
Description	ZAP passive scanner and PTK both confirmed that the Content-Security-Policy HTTP response header is absent across the entire application. CSP is a critical browser-enforced security mechanism that restricts which resources (scripts, styles, images) can be loaded. Its absence significantly amplifies the impact of XSS vulnerabilities already identified. CWE-693 | OWASP A05:2021 Security Misconfiguration.
Target	All pages — demo.testfire.net (confirmed on 5 instances by ZAP)
Risk Rating	Medium (10)
Attack techniques	Without CSP, any successful XSS injection (V-01) has unrestricted ability to load external scripts, exfiltrate data to attacker-controlled servers, perform UI redirection, and execute arbitrary JavaScript. CSP acts as a last line of defence that limits XSS blast radius.
Countermeasures	Add a Content-Security-Policy response header with a strict policy: Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'. This can be configured in Apache/Nginx or via a Java servlet filter. Use CSP nonces for inline scripts rather than 'unsafe-inline'.
Reference	MDN CSP Guide: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP | OWASP CSP Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html | CWE-693
Screenshot	

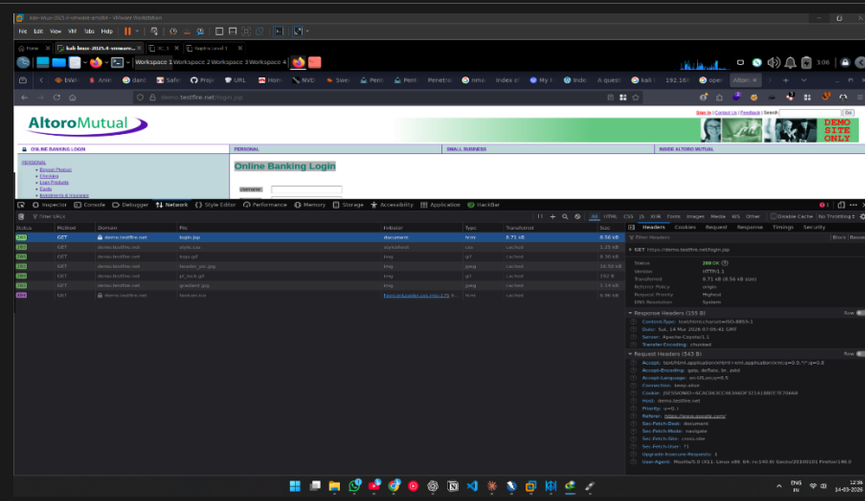
V-06	Missing Anti-Clickjacking Header (X-Frame-Options)
Description	ZAP passive scanner confirmed the absence of the X-Frame-Options HTTP response header on all tested pages. Without this header, any external website can embed demo.testfire.net pages inside an <iframe>. This enables clickjacking attacks where a victim believes they are clicking legitimate UI elements but are actually interacting with a hidden frame. CWE-1021 | OWASP A05:2021.
Target	All pages — demo.testfire.net (confirmed on 5 instances by ZAP)
Risk Rating	Medium (10)
Attack techniques	An attacker creates a malicious webpage that loads the AltoroMutual login page in a transparent iframe overlaid on top of an attractive decoy page. Victims unknowingly type their credentials into the real login form (visible through the iframe), which are then captured.
Countermeasures	Add the HTTP response header: X-Frame-Options: DENY (or SAMEORIGIN if framing within the same domain is required). Additionally, include frame-ancestors 'none' in the Content-Security-Policy header as a modern replacement. Configure in Apache/Nginx or via Java FilterRegistrationBean.
Reference	OWASP Clickjacking Defense Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Clickjacking_Defense_Cheat_Sheet.html | MDN X-Frame-Options
Screenshot	

V-07	Sub Resource Integrity (SRI) Attribute Missing
Description	ZAP identified that externally loaded resources (JavaScript and CSS files loaded from external CDNs or third-party origins) lack Subresource Integrity (SRI) integrity attributes. SRI allows browsers to verify that files fetched from external servers have not been tampered with by requiring a cryptographic hash in the HTML tag. CWE-353 | OWASP A05:2021.
Target	http://demo.testfire.net — External script/style references
Risk Rating	Medium (10)
Attack techniques	If an external CDN or script host is compromised, an attacker can modify the JavaScript file to inject malicious code. Without SRI, the victim's browser loads and executes the modified script without any warning. This is a supply chain attack vector.
Countermeasures	Add integrity and crossorigin attributes to all <script> and <link> tags that load external resources. Example: <script src="https://cdn.example.com/lib.js" integrity="sha384-..." crossorigin="anonymous">. Generate hashes using the SRI Hash Generator at https://www.srihash.org/.
Reference	MDN Sub Resource Integrity: https://developer.mozilla.org/en-US/docs/Web/Security/Subresource_Integrity | OWASP A05:2021
Screenshot	

V-08	Server Banner Discloses Software Version
Description	All HTTP responses from demo.testfire.net include the Server: Apache-Coyote/1.1 response header, confirming the exact application server (Apache Tomcat) and its internal component name. This was confirmed by ZAP passive scanner, PTK, and browser DevTools. The ISO-8859-1 charset in the Content-Type header further reveals legacy platform details. CWE-200 | OWASP A05:2021.
Target	HTTP Response Header — Server: Apache-Coyote/1.1 (all responses)
Risk Rating	Medium (10)
Attack techniques	Knowledge of the exact server software and version allows an attacker to look up CVE databases for known vulnerabilities specific to Apache-Coyote/Tomcat. This dramatically reduces research time needed to identify exploitable weaknesses. Combined with the open-source codebase link in the footer, the entire attack surface is effectively documented.
Countermeasures	Suppress the Server response header in Tomcat by adding <Connector server=" " /> in server.xml (replace with a blank or generic value). Remove X-Powered-By headers. Remove the GitHub source code link from the application footer. Update Tomcat to a current supported version and apply all security patches.
Reference	CWE-200: https://cwe.mitre.org/data/definitions/200.html | Tomcat Security: https://tomcat.apache.org/tomcat-9.0-doc/security-howto.html
Screenshot	

V-09	Session Cookie Without SameSite Attribute
Description	ZAP passive scanner identified that the JSESSIONID session cookie is set without the SameSite attribute (confirmed 3 instances). Without SameSite, the cookie is sent with all cross-site requests, enabling CSRF attacks and session token leakage in cross-origin contexts. CWE-1275 | OWASP A07:2021 Identification and Authentication Failures.
Target	JSESSIONID cookie — Set on all authenticated responses
Risk Rating	Low (7)
Attack techniques	An attacker can craft cross-origin requests (from a malicious site) that include the JSESSIONID cookie, enabling CSRF attacks. Combined with the missing CSRF token (V-03), this compounds the risk significantly. SameSite=None without Secure also risks cookie leakage over HTTP.
Countermeasures	Set the SameSite=Strict attribute on the JSESSIONID cookie. In Java/Tomcat, configure <Context> with HttpOnly and Secure flags in web.xml, or programmatically via SessionCookieConfig. The cookie should be: Set-Cookie: JSESSIONID=...; HttpOnly; Secure; SameSite=Strict.
Reference	OWASP Session Management Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html | CWE-1275

V-10	Cross-Domain JavaScript Source File Inclusion
Description	ZAP identified that the application includes JavaScript files sourced from external, cross-domain origins without SRI validation. This finding is closely related to V-07 (SRI Missing) but specifically flags the cross-origin inclusion pattern itself as a separate risk vector. CWE-829 | OWASP A08:2021 Software and Data Integrity Failures.
Target	http://demo.testfire.net — Script tags referencing external domains
Risk Rating	Low (7)
Attack techniques	If the external domain hosting the JavaScript file is compromised or the DNS record is hijacked, an attacker can serve malicious JavaScript to all visitors of demo.testfire.net. The browser will execute it with full access to the page DOM, cookies, and user input.
Countermeasures	Audit all external script inclusions. Where possible, self-host critical JavaScript libraries rather than relying on external CDNs. For any remaining external inclusions, add SRI integrity hashes (see V-07). Implement CSP script-src directives to whitelist only approved script origins.
Reference	OWASP A08:2021 — Software and Data Integrity Failures | CWE-829: https://cwe.mitre.org/data/definitions/829.html

V-11	X-Content-Type-Options Header Missing
Description	ZAP confirmed that the X-Content-Type-Options: nosniff HTTP response header is absent across all tested pages. Without this header, browsers may perform MIME-type sniffing — interpreting a response as a different content type than declared. This can allow attackers to serve malicious content disguised as benign files. CWE-693 | OWASP A05:2021.
Target	All pages — demo.testfire.net (confirmed on 5 instances by ZAP)
Risk Rating	Low (7)
Attack techniques	An attacker uploads or links to a file that is served with an incorrect MIME type (e.g. an HTML file served as text/plain). Without nosniff, the browser may execute it as HTML/JavaScript, enabling stored XSS or content injection attacks even in scenarios where the application believes it is serving safe content.
Countermeasures	Add the HTTP response header: X-Content-Type-Options: nosniff to all responses. This is a one-line configuration change in Apache (Header always set X-Content-Type-Options nosniff) or Nginx (add_header X-Content-Type-Options nosniff). In Java, add via a servlet filter.
Reference	OWASP Secure Headers: https://owasp.org/www-project-secure-headers/ | MDN X-Content-Type-Options | CWE-693
Screenshot	

V-12	Missing Referrer-Policy Header
Description	PTK confirmed the absence of a Referrer-Policy HTTP response header across the application. Without this header, the full URL of the current page (including query parameters) is transmitted in the Referer header to third-party resources. For a banking application, this can leak sensitive URL parameters to external servers. CWE-200, CWE-525 | OWASP A05:2021.
Target	All pages — demo.testfire.net (confirmed by PTK on multiple responses)
Risk Rating	Low (7)
Attack techniques	If a user navigates from a sensitive URL (e.g. one containing an account ID or token in the query string) to an external resource, the full referrer URL is sent to that third party in the HTTP Referer header. This leaks user session context and navigation patterns to external analytics or CDN services.
Countermeasures	Add the HTTP response header: Referrer-Policy: strict-origin-when-cross-origin (recommended) or no-referrer for maximum privacy. This prevents the full URL from being leaked to cross-origin destinations while retaining the origin for same-origin requests.
Reference	MDN Referrer-Policy: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy | CWE-200, CWE-525

V-13	Swagger UI Publicly Accessible Without Authentication
Description	PTK and ZAP spider both confirmed that a Swagger UI interface is publicly accessible at /swagger/index.html with no authentication required. This exposes the complete REST API documentation including all endpoints, HTTP methods, request parameters, and expected response schemas to any anonymous visitor. CWE-200 | OWASP A05:2021.
Target	https://demo.testfire.net/swagger/index.html
Risk Rating	Informational (5)
Countermeasures	Remove Swagger UI from the production deployment entirely. If API documentation is required, restrict access to authenticated administrators or internal IP ranges only. Consider using API gateway-level access controls.
Reference	OWASP API Security Top 10: https://owasp.org/www-project-api-security/ | CWE-200

V-14	Information Disclosure — Suspicious Comments in HTML Source
Description	ZAP passive scanner identified 6 instances of suspicious comments embedded in the HTML source code across multiple pages. The page source also contains comment blocks BEGIN HEADER / END HEADER / BEGIN FOOTER / END FOOTER along with developer-facing structure notes. The footer explicitly links to the full open-source GitHub repository (github.com/AppSecDev/AltoroJ). CWE-200.
Target	HTML source — multiple pages (6 instances confirmed by ZAP)
Risk Rating	Informational (5)
Countermeasures	Strip all developer comments from production HTML output. Remove the GitHub repository link from the application footer. Implement a build pipeline step that strips HTML comments before deployment. Review all JSP template files for hardcoded credentials, internal IP addresses, or debug output.
Reference	CWE-200: Information Exposure | OWASP Testing Guide: WSTG-INFO-05

V-15	Session Management Token Identified
Description	ZAP passive scanner identified the session management mechanism: a JSESSIONID cookie using Java's default session token. The token is set without the Secure flag in HTTP responses, meaning it can be transmitted over unencrypted connections. Combined with the mixed HTTP/HTTPS content issue, this creates a path to session token interception. CWE-311 | OWASP A02:2021.
Target	JSESSIONID cookie — Set-Cookie response header
Risk Rating	Informational (5)
Countermeasures	Set the Secure flag on the JSESSIONID cookie to ensure it is only transmitted over HTTPS. Enforce HTTPS site-wide via HSTS (Strict-Transport-Security header). Regenerate session tokens after authentication. Implement session timeout policies.
Reference	OWASP Session Management Cheat Sheet | CWE-311: https://cwe.mitre.org/data/definitions/311.html